BIUST

Faculty of Engineering

Electrical, Computer and Telecommunications Department

LAB 03: 4x3 Keypad and 16x2 LCD Interfacing with PIC16 microcontroller

Overview

In this lab, you will learn to interface a keypad and an LCD with a pic microcontroller. The lab is a continuation of lab02. In this lab, we add a 4x3 keypad to the already integrated 16x2 LCD (Liquid Crystal Display) to the PIC microcontroller. *The code remains the same, we simple add a function for the 4x3 keypad.* After completion of this lab, you will be able to control displayed data on an LCD using a keypad and the MPLAB XC8 Compiler.

Objectives:

- To identify pinout of PIC16F877A
- To identify 16x2 LCD pins
- To identify 4x3 keypad pins
- To design a keypad and LCD interfaced with PIC16 Circuit
- To design a flowchart model and Pseudocode for the 4x3 keypad and 16x2 LCD Interfacing with PIC16 microcontroller
- To write a program that displays characters on the 16x2 LCD controlled by a keypad

Materials list

- U1: Microcontroller PIC16F877A
- LCD1: 16x2 LCD
- 4x3 keypad

- RV1: Variable Resistor
- X1: Crystal oscillator
- C1: 22p x 2 Capacitors

Task 1: Creating a project on MPLab x

1. Open MPLab x.
2. Create a new project
3. Select categories: Microchip Embedded and projects: Standalone projects. Click
4. on next.
5. Select the device type as PIC16F877A. Under tools select simulator. You may search for the microcontroller by entering 877A under device.
6. Select the compiler: xc8
7. Give your project a name and folder. Set it as the main project, you must always set the project you are working on as the main project.
8. Click on finish.

You are done starting a new project. Next step is '**c coding**'.

1. Right click on 'Header Files', select new, then select xc8_header.h
2. You may rename you file to anything you desire, but it is better to name it as lcd.h
3. Open lcd.h. You should see something like this:

```
/*
 * File:
 * Author:
 * Comments:
 * Revision history:
 */

// This is a guard condition so that contents of this file are not included
// more than once.
#ifndef XC_HEADER_TEMPLATE_H
#define XC_HEADER_TEMPLATE_H

#include <xc.h> // include processor files - each processor file is guarded.

// TODO Insert appropriate #include <>

// TODO Insert C++ class definitions if appropriate

// TODO Insert declarations
```

Figure showing the Header file called lcd.h

4. Copy and paste lcd.h header code lines below. The TAs will explain the code for you.

Next:

5. Right click on ‘Source Files’, select new, then select main.c
6. You may rename you file to anything you desire, but it is better to name it as main.c
7. Open main.c. You should see something like this:

```c
/*
 * File:   newmain.c
 * Author: Ditshegon
 *
 * Created on September 2, 2024, 2:50 PM
 */


#include <xc.h>

void main(void) {
    return;
}

```

Figure showing main.c

8. On the file menu, click on ‘Production’, select ‘Set Configuration Bits’
9. Figure showing how to select ‘Set Configuration Bits’
10. The FOSC is the the clock that controls the execution of the instructions. Make it external. Turn off everything else.
11. Generate the Source Code to Output.
12. Define the inputs and outputs for the keypad after the LCD at the point:

```c
#define RS RD2
#define EN RD3
#define D4 RD4
#define D5 RD5
#define D6 RD6
#define D7 RD7

#define R1 RB4
#define R2 RB5
#define R3 RB6
#define R4 RB7
#define C1 RB2
#define C2 RB1
```

#define C3 RB0

13. Keep the library as '#include <xc.h>'
14. Define the external clock as 8MHz: #define _XTAL_FREQ 8000000

The lab technicians will now help you to write the 'void main()' method. The method allows a user to write characters on the LCD using the PIC microcontroller. Once done, you will clean and build the main project. This will produce a file with an extension called .hex.

Things to do:

1. Design the flowchart model and the Pseudocode for the 4x3 keypad, 16x2 LCD Interfacing with PIC16 microcontroller
2. Take pictures of the major steps done.
3. You must copy and paste the code you did with the technicians in the lab.

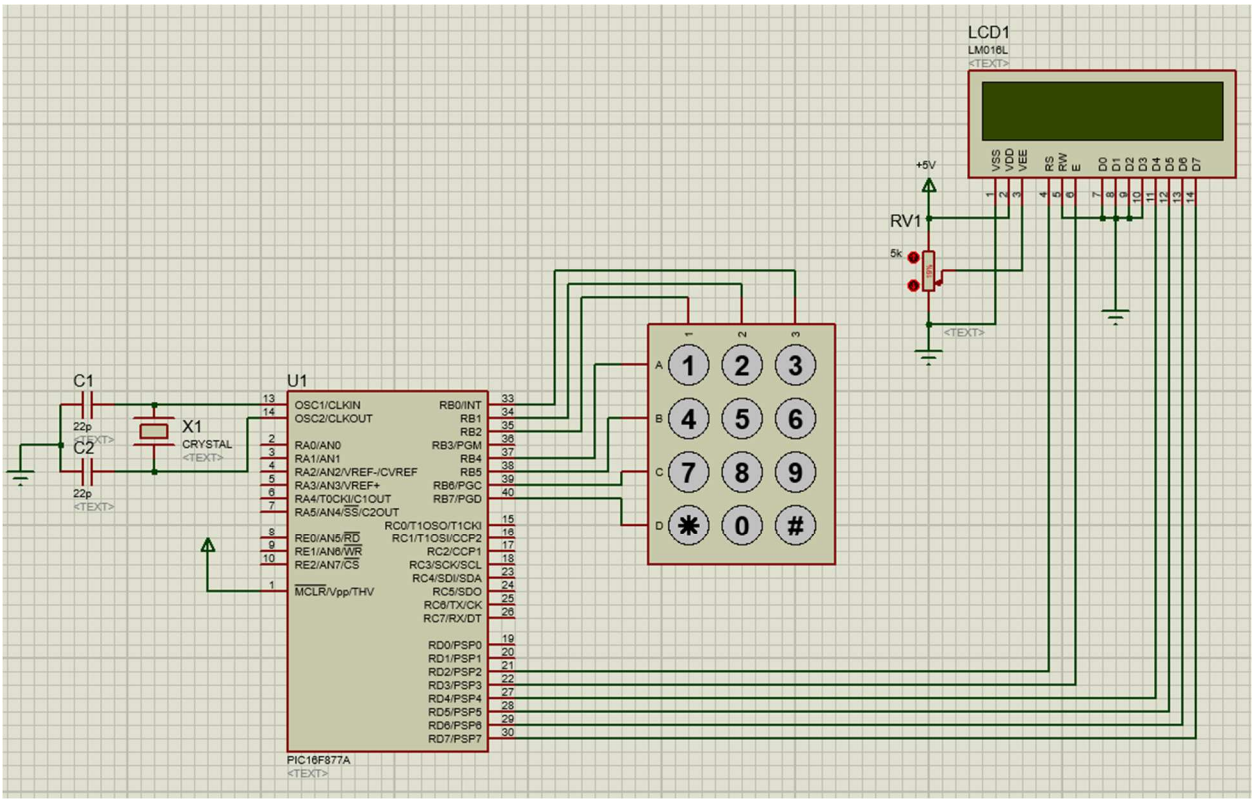
Note: The .hex is located at C:\Users\...\MPLABXProjects\LCDInterface.X\dist\default\production

Task 2: Using Proteus to simulate the Hardware

1. Create a new project in the Proteus software.
2. Create the project name, the path where it will be located. Click next.
3. Create a DEFAULT schematic template. Click next.
4. Do not create a PCB layout. Click next.
5. Select 'No Firmware Project'. Click next.
6. Click on 'Finish'.

Your document has been created. Now, it is time to populate it with components. The idea is to simulate the code in the schematic shown below.

Circuit diagram



Header file for the LCD.h XC8 program

```
/* Microchip Technology Inc. and its subsidiaries.  You may use this software
 * and any derivatives exclusively with Microchip products.
 *
 * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS".  NO WARRANTIES, WHETHER
 * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
 * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
 * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS,
COMBINATION
 * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
 *
 * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
 * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
 * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
HAS
 * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE.  TO THE
 * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
 * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
 * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
```

```
 *
 * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF
THESE
 * TERMS.
 */

/*
 * File:
 * Author:
 * Comments:
 * Revision history:
 */

// This is a guard condition so that contents of this file are not included
// more than once.
//LCD Functions Developed by electroSome

void Lcd_Port(char a)
{
	if(a & 1)
		D4 = 1;
	else
		D4 = 0;
	if(a & 2)
		D5 = 1;
	else
		D5 = 0;
	if(a & 4)
		D6 = 1;
	else
		D6 = 0;

	if(a & 8)
		D7 = 1;
	else
		D7 = 0;
}
```

```
void Lcd_Cmd(char a)
{
        RS = 0;         // => RS = 0
        Lcd_Port(a);
        EN  = 1;         // => E = 1
    __delay_ms(4);
    EN  = 0;         // => E = 0
}

void Lcd_Clear()
{
        Lcd_Cmd(0);
        Lcd_Cmd(1);
}

void Lcd_Set_Cursor(char a, char b)
{
        char temp,z,y;
        if(a == 1)
        {
          temp = 0x80 + b - 1;
                z = temp>>4;
                y = temp & 0x0F;
                Lcd_Cmd(z);
                Lcd_Cmd(y);
        }
        else if(a == 2)
        {
                temp = 0xC0 + b - 1;
                z = temp>>4;
                y = temp & 0x0F;
                Lcd_Cmd(z);
                Lcd_Cmd(y);
        }
}

void Lcd_Init()
```

```
{
  Lcd_Port(0x00);
   __delay_ms(20);
  Lcd_Cmd(0x03);
            __delay_ms(5);
  Lcd_Cmd(0x03);
            __delay_ms(11);
  Lcd_Cmd(0x03);
  ////////////////////////////////////////////////////
  Lcd_Cmd(0x02);
  Lcd_Cmd(0x02);
  Lcd_Cmd(0x08);
  Lcd_Cmd(0x00);
  Lcd_Cmd(0x0C);
  Lcd_Cmd(0x00);
  Lcd_Cmd(0x06);
}

void Lcd_Write_Char(char a)
{
   char temp,y;
   temp = a&0x0F;
   y = a&0xF0;
   RS = 1;            // => RS = 1
   Lcd_Port(y>>4);            //Data transfer
   EN = 1;
   __delay_us(40);
   EN = 0;
   Lcd_Port(temp);
   EN = 1;
   __delay_us(40);
   EN = 0;
}

void Lcd_Write_String(char *a)
{
            int i;
```

```
        for(i=0;a[i]!='\0';i++)
          Lcd_Write_Char(a[i]);
}

void Lcd_Shift_Right()
{
        Lcd_Cmd(0x01);
        Lcd_Cmd(0x0C);
}

void Lcd_Shift_Left()
{
        Lcd_Cmd(0x01);
        Lcd_Cmd(0x08);
}
```

XC8 program for the

```
/*
 * File:   main.c
 * Author: Nonofo
 *
 * Created on September 9, 2024, 10:34 PM
 */
// BEGIN CONFIG
#pragma config FOSC = EXTRC // Oscillator Selection bits (RC oscillator)
#pragma config WDTE = OFF // Watchdog Timer Enable bit (WDT enabled)
#pragma config PWRTE = OFF // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = OFF // Brown-out Reset Enable bit (BOR disabled)
#pragma config LVP = OFF // Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit
(RB3 is digital I/O, HV on MCLR must be used for programming)
#pragma config CPD = OFF // Data EEPROM Memory Code Protection bit (Data EEPROM code
protection off)
#pragma config WRT = OFF // Flash Program Memory Write Enable bits (Write protection off; all
program memory may be written to by EECON control)
#pragma config CP = OFF // Flash Program Memory Code Protection bit (Code protection off)
//END CONFIG

#define _XTAL_FREQ 8000000

#define RS RD2
#define EN RD3
#define D4 RD4
#define D5 RD5
#define D6 RD6
```

```
#define D7 RD7

#define R1 RD4
#define R2 RD5
#define R3 RD6
#define R4 RD7
#define C1 RD2
#define C2 RD1
#define C3 RD0

#include <xc.h>
#include "lcd.h"

void keypad()
{
    Complete this method with Technicians, IT and TA at the lab.
}

int main()
{
     Complete this method with Technicians, IT and TA at the lab.
}
```

SIMULATIONS

1. Write the code in MPLAB X IDE and execute successfully so as to generate a HEX file. TECHNICIAN and TA will assist.
2. Design the flowchart model and the Pseudocode for the 4x3 keypad, 16x2 LCD Interfacing with PIC16 microcontroller
3. Draw the circuit in PROTEUS starting with a blank script. Make sure to use PIC16F877A microcontroller. The HEX file must be loaded within the microcontroller so as the circuit can run successfully. TECHNICIAN and TA will assist.
4. PROTUES is used to visually check the circuit and code. Once all works, you are now ready to move to REAL COMPONENTS.

REAL COMPONENTS

5. TECHNICIAN and TA will assist will loading the HEX file in microcontroller.
6. Connect the circuit on BREADBOARD. Make sure to connect 8 000 000 Hz external crystal oscillator across TWO 22p CAPACITORS.
7. Connect POWER SUPPLY and check circuit.

You must take PICTURES and write procedures of all steps in your lab report.